UNIVERSITY OF ESWATINI

CSC392: Principles of Software Engineering I — Mini Project

SOFTWARE DESIGN DOCUMENT

Universal Exam Card System

Field	Detail
Names	*Sharoon Nqobile Shoulder* *Bandile Lokothwayo* *Gaute Banda* *Senamiso Masango*
Date	14/05/2026
Institution	University of Eswatini

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) describes the architecture, system design, components, interfaces, and data structures of the Universal Exam Card System. It translates the functional and non-functional requirements from the SRS into a detailed blueprint that will guide the implementation phase. The primary audience includes software developers, testers, and project maintainers.

1.2 Scope

The Universal Exam Card System automates student identity verification and exam script tracking at the University of Eswatini using QR code technology.

Goals and Objectives:

- Generate a single digital Universal Exam Card consolidating all registered modules.
- Enable real-time secure identity verification and attendance marking via Android app.
- Digitally log script collection with invigilator accountability.
- Provide real-time dashboards and reports for administrators and lead invigilators.

Benefits: Reduced impersonation, elimination of lost scripts, improved exam integrity, and better accountability.

In Scope: Backend API, React web dashboard, Android mobile app (with offline support), MySQL database.

Out of Scope: Physical card printing, biometric authentication, integration with financial/HR systems.

1.3 Overview

This document follows the university SDD template (adapted from IEEE Std 1016-2009). This document is organized as follows:

- Section 2 provides a high-level system overview.
- Section 3 describes the system architecture and decomposition.
- Section 4 details the data design.
- Section 5 presents component-level design.
- Section 6 covers the human interface design.

- Section 7 provides traceability to SRS requirements.
- Section 8 contains supporting appendices.

1.4 Reference Material

- SRS Version 1.0 – Universal Exam Card System (01 May 2026)
- IEEE Std 1016-2009-Recommended Practice for Software Design Descriptions

1.5 Definitions and Acronyms

Acronyms	Definitions
UEC	Universal Exam Card – Digital identity token with QR code.
QR Code	Quick Response code containing encrypted student verification data.
Scan Log	Record of verification or script collection event.
JWT	JSON Web Token – Used for authentication
Lead Invigilator	Supervisor with dashboard access.

2. SYSTEM OVERVIEW

The Universal Exam Card System replaces error-prone manual processes with a secure digital solution. Students access their UEC via a web portal. Invigilators use an Android app to scan QR codes for real-time verification, attendance marking, and script logging. Administrators and lead invigilators monitor live attendance and generate reports through a web dashboard. The system supports offline functionality on mobile with later synchronization.

3. Design Overview

This section provides a high-level summary of the design approach, stakeholders, key design concerns, selected design viewpoints, notations used, and design constraints. It establishes the foundation for understanding the detailed design presented in subsequent sections.

3.1 Design Stakeholders

The following stakeholders have an interest in the design of the Universal Exam Card System:

Stakeholder	Role / Involvement	Key Interests / Concerns
Students	End users of the Universal Exam Card	Privacy, security of personal data, ease of accessing their digital exam card
Invigilators	Primary users of the Android mobile application	Fast scanning, reliable offline functionality, simple and intuitive interface, quick feedback
Lead Invigilators	Supervisors during examinations	Real-time attendance visibility, anomaly detection, script collection monitoring
Administrators	System administrators and exam coordinators	Dashboard oversight, report generation, auditability, system reliability
Developers / Maintainers	Implementation and future maintenance team	Modularity, code maintainability, scalability, clear component interfaces
University Management	Project sponsors and policy enforcers	Exam integrity, reduction of impersonation and malpractice, accountability, data security

3.2 Design Concerns

The design addresses the following critical concerns derived from the SRS:

- **Security & Integrity**: Prevention of impersonation through encrypted QR codes and strong authentication.
- **Reliability in Challenging Environments**: Support for offline operation in examination venues with poor or no network connectivity.
- **Performance & Scalability**: Ability to handle high concurrency (hundreds to thousands of simultaneous scans) during peak examination periods.
- **Real-time Monitoring**: Provision of live attendance and script collection status to lead invigilators and administrators.
- **Usability**: Simple, scanner-centric interface for invigilators who may not be technically proficient.
- **Auditability & Traceability**: Complete logging of all verification and script collection actions for accountability.

- **Data Consistency**: Proper synchronization between offline mobile operations and the central database.
- **Maintainability**: Clear separation of concerns to support future enhancements within limited resources.

3.3 Selected Design Viewpoints

- Context Viewpoint: Defines the system boundary and its interactions with external entities (students, invigilators, administrators, and external systems).
- Composition Viewpoint: Describes the overall system structure, major subsystems, and their relationships.
- Logical / Behavioural Viewpoint: Presents functional decomposition, workflows, and dynamic behaviour of the system.
- Interface Viewpoint: Details the external interfaces, especially the RESTful API and WebSocket communication.
- Information Viewpoint: Describes the data structures, entities, relationships, and storage mechanisms.
- Deployment / Physical Viewpoint: (Implied) Describes how the system is deployed across web, mobile, and database tiers.

3.4 Design Languages and Notations

The following notations and languages are used to describe the design:

Design Aspect	Notation / Language Used
Architecture Diagrams	UML Component, Deployment & Package diagrams
Behavioural Modelling	UML Activity Diagrams, Sequence Diagrams
Data Modelling	Entity-Relationship (ER) Diagrams
Data Dictionary	Tabular format with attributes, keys, and descriptions
API Specification	RESTful endpoint definitions with JSON examples
Algorithms & Logic	Structured Pseudocode
User Interface	Screen mock-ups and textual descriptions
Database Schema	MySQL DDL conventions

3.5 Design Constraints

The following constraints significantly influenced the design decisions:

- **Academic Project Timeline**: Must be completed within one semester by undergraduate students.
- **Technology Stack**: Limited to technologies familiar to the development team (Node.js, React, Java for Android, MySQL).
- **Monolithic Architecture**: Chosen over microservices due to time and complexity constraints.
- **Infrastructure**: Must run on university-provided or low-cost cloud/server resources.
- **Mobile Platform**: Android only, with mandatory offline support using Room SQLite and WorkManager.
- **No Biometrics**: Physical card printing and biometric authentication are out of scope.
- **Security Limitations**: Use of JWT, BCrypt, and TLS without advanced enterprise security tools.

3.6 Design Rationale Summary

A 3-tier Client-Server architecture (Presentation → Application → Data) was selected to achieve a clean separation of concerns, improve maintainability, and support multiple client types (Web + Mobile).

The combination of RESTful APIs for core operations and WebSockets for real-time updates provides the right balance between simplicity and responsiveness. A monolithic backend was preferred for faster development and easier deployment within project constraints, while still allowing future scaling through Docker containerization.

Offline support using local SQLite with intelligent synchronization was made a core design priority due to the unreliable network conditions common in large examination venues

4. SYSTEM ARCHITECTURE

4.1 Architectural Design

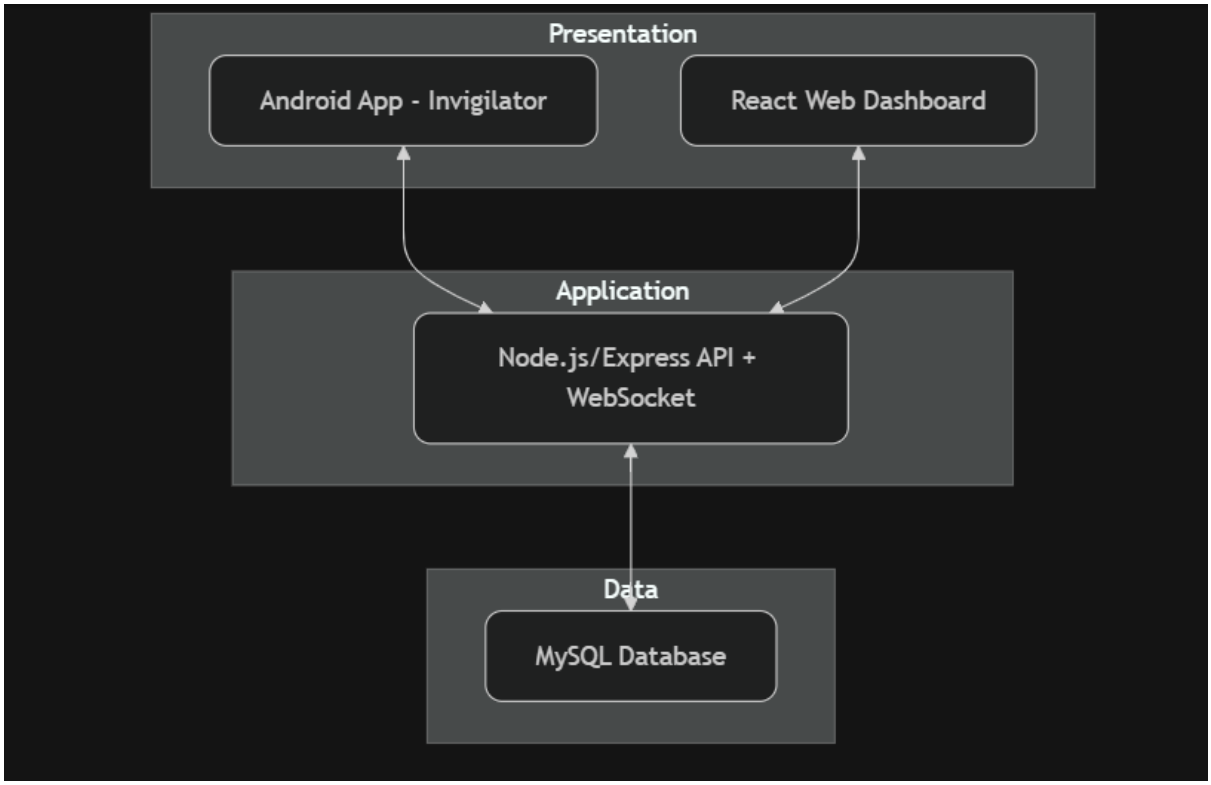
The system follows a 3-tier Client-Server Architecture:

- **Presentation Tier**: React.js Web Dashboard + Android Application (Java)
- **Application Tier**: Node.js/Express RESTful API + WebSocket Server
- **Data Tier**: MySQL Relational Database

Major Subsystems and Responsibilities include:

- **Authentication Subsystem**: User login and JWT management.
- **Exam Card Management Subsystem**: Generation and QR encoding.
- **Verification & Logging Subsystem**: QR scanning, validation, attendance & script logging.
- **Monitoring & Reporting Subsystem**: Real-time dashboard and report generation.

Architecture Diagram:



The subsystems collaborate via REST APIs for standard operations and WebSockets for real-time updates.

4.2 Decomposition Description

Functional Decomposition:

1. Generate Universal Exam Card
2. Authenticate Users
3. Scan & Verify Student
4. Mark Attendance & Log Script Collection
5. Monitor Dashboard (Real-time)
6. Generate Reports

5. DATA DESIGN

5.1 Data Description

The information domain is transformed into a relational database model using MySQL. Data is normalized to 3NF to reduce redundancy and ensure integrity.

Storage: All persistent data is stored in a MySQL relational database. QR code data is stored as encrypted strings. Scan logs are append-only for auditability. The Android app uses local Room SQLite database for offline storage and synchronizes with the central MySQL when online using WorkManager.

Processing:

- Student and exam data are joined during verification.
- Timestamps and foreign keys maintain audit trails.
- Indexes on student_id, session_id, and timestamp ensure fast queries during high-load exam periods.

Major Entities:

- Students, Exam Cards, Exam Sessions, Scan Logs, Invigilators.

5.2 Data Dictionary

The following tables describe all entities in the exam_portal MySQL database. Keys: PK = Primary Key, FK = Foreign Key.

Entity: Students			
Stores unique identification and credential information for students.			
Attribute	**Data Type**	**Key**	**Description**
student_id	varchar(20)	PK	Unique student number (e.g., 2026001).
first_name	varchar(50)		Student's first name.
last_name	varchar(50)		Student's last name.
password	varchar(255)		Hashed credential for system access.
program	varchar(50)		Degree programme the student is enrolled in (e.g., BSc Computer Science).
created_at	timestamp		Record creation timestamp, auto-populated on insert.
image_url	varchar(255)		URL to the student's profile photo, used during identity verification.

Entity: Courses			
Defines the academic subjects available for examination.			
Attribute	**Data Type**	**Key**	**Description**
course_code	varchar(20)	PK	Unique alphanumeric code identifying the course (e.g., CSC101).
course_name	varchar(100)		The full descriptive title of the course.
credits	int		Credit weight of the course. Defaults to 1.

department_id	varchar(20)	FK	References departments — links the course to its offering department.

Entity: Departments			
Represents the academic departments that own and offer courses.			
Attribute	**Data Type**	**Key**	**Description**
department_id	varchar(20)	PK	Unique department identifier (e.g., DEPT-CS).
department_name	varchar(100)		Full name of the department (e.g., Computer Science).
seat_prefix	varchar(10)		Short prefix used when generating seat numbers for students in this department.

Entity: Enrollments			
A dynamic table linking students to the specific courses they are registered for in a given semester.			
Attribute	**Data Type**	**Key**	**Description**
student_id	varchar(20)	PK / FK	References students — composite PK with course_code.
course_code	varchar(20)	PK / FK	References courses — composite PK with student_id.

academic_year	varchar(10)		The academic year of enrollment (e.g., 2026/2027).
semester	tinyint		Semester number within the academic year.
seat_number	int		Assigned examination seat number for this enrollment.
exam_status	varchar(20)		Current exam sitting status: 'Not Written' or 'Written'. Defaults to 'Not Written'.
ca_mark	decimal(5,2)		Continuous assessment score carried into this enrollment record. Defaults to 0.00.

Entity: Exam_Schedules			
The master timetable linking courses to their examination times and locations.			
Attribute	**Data Type**	**Key**	**Description**
course_code	varchar(20)	PK / FK	References courses — the course whose exam is scheduled. Part of composite PK.
exam_date	date		The calendar date on which the exam takes place.
venue	varchar(100)		Name of the physical examination hall or venue (e.g., Sports Emporium).
academic_year	varchar(10)		The academic year this schedule entry belongs to.
semester	tinyint		Semester number for this scheduled exam.

start_time	time		Scheduled start time. Defaults to 09:00.
end_time	time		Scheduled end time. Defaults to 14:00.

Entity: Invigilators

Stores details for staff members responsible for supervising examinations.

Attribute	Data Type	Key	Description
invigilator_id	varchar(20)	PK	Unique identifier for the invigilator.
first_name	varchar(50)		Invigilator's first name.
middle_name	varchar(50)		Invigilator's middle name.
last_name	varchar(50)		Invigilator's last name.
email	varchar(100)		Staff email address used for login.
password	varchar(255)		Hashed credential for system access.
phone	varchar(20)		Contact phone number (optional).
department_id	varchar(20)	FK	References departments — the invigilator's home department.
role	enum		Access level: 'invigilator', 'chief_invigilator', or 'admin'.
is_active	tinyint(1)		Boolean flag — 1 = active account, 0 = deactivated.
created_at	timestamp		Account creation timestamp, auto-populated on insert.

Entity: CA_Marks			
Stores continuous assessment scores submitted per student per course.			
Attribute	**Data Type**	**Key**	**Description**
student_id	varchar(20)	PK / FK	References students — composite PK with course_code.
course_code	varchar(20)	PK / FK	References courses — composite PK with student_id.
ca_score	decimal(5,2)		The student's continuous assessment score for the course.
status	enum		CA sitting status: 'Not written', 'Written', or 'Absent'. Defaults to 'Not written'.

Entity: Seat_Allocations			
Records the physical seat assigned to each student for a specific examination.			
Attribute	**Data Type**	**Key**	**Description**
student_id	varchar(20)	PK / FK	References students — part of composite PK.
course_code	varchar(20)	PK / FK	References courses — part of composite PK.
exam_date	date		The exam date this seat allocation is valid for.

seat_number	varchar(15)		Assigned seat label (e.g., CSC-001), generated using the department seat prefix.
department_id	varchar(20)	FK	References departments — used to group and prefix seating by department.
allocated_at	timestamp		Timestamp of when the seat was assigned. Auto-populated.

Entity: Scan_Logs			
Audit trail for all QR code interactions during the invigilation process.			
Attribute	**Data Type**	**Key**	**Description**
log_id	int (AUTO)	PK	Auto-generated unique identifier for each scan event.
student_id	varchar(20)	FK	References students — the student whose QR code was scanned.
invigilator_id	varchar(20)	FK	References invigilators — the staff member who performed the scan.
course_code	varchar(20)	FK	References courses — the exam for which the scan was performed.
scan_time	timestamp		Exact date and time of the scan transaction. Defaults to current timestamp.
venue	varchar(100)		The physical location where the scan occurred.
action	varchar(50)		Type of scan event (e.g., 'checked_in'). Defaults to 'checked_in'.

script_qr_code	varchar(50)		The QR code value scanned from the student's exam script for tracking purposes.

Entity: Venues			
Physical locations where examinations are held.			
Attribute	**Data Type**	**Key**	**Description**
venue_id	int (AUTO)	PK	Auto-generated unique identifier for the venue.
venue_name	varchar(100)		The name or number of the examination hall (e.g., MPH, Sports Emporium).

5.3 Interface Design (API Specification)

The system utilizes a RESTful API architecture to facilitate communication between the Mobile Client and the MySQL Database. All requests must be made over HTTPS.

5.3.1 Authentication Flow

The system uses JWT (JSON Web Tokens) for secure stateless authentication.

1. Login: The user (Invigilator) sends credentials to /api/auth/login.
2. Token Issuance: Upon validation, the server returns a signed JWT.
3. Authorized Requests: The mobile app must include this token in the Authorization header as a Bearer token for all subsequent requests (e.g., QR verification).

5.3.2 Endpoint List & Definitions

Method	Endpoint	Description	Authorisation Required
POST	/api/auth/login	Validates invigilator credentials and returns a JWT.	No
GET	/api/sync/pre-fetch	Downloads student/enrolment list for offline use.	Yes

POST	/api/verify-qr	Validates scanned student QR data against the schedule	Yes
POST	/api/sync/bulk-upload	Uploads locally stored scan logs once online	Yes

5.3.3 Request/Response Formats

5.3.3.1 POST /api/verify-qr

This is the primary endpoint used during the examination process.

Request Body (JSON):

```
{
 "student_id": 123456,
 "course_code": "STA301",
 "venue_id": 10,
 "scan_timestamp": "2026-05-15T09:00:00Z",
 "invigilator_id": 55
}
```

Success Response (200 OK):

```
{
 "status": "success",
 "message": "Student verified and attendance logged.",
 "data": {
  "student_name": "John Doe",
  "course_name": "Time Series Analysis"
 }
}
```

Error Response (403 Forbidden):

```
{
 "status": "error",
 "error_code": "NOT_ENROLLED",
 "message": "Student is not enrolled for this specific course exam."
}
```

5.3.3.2 POST /api/sync/bulk-upload

Used to synchronize data collected during Offline Mode.

Request Body (JSON):

```
{
 "batch_id": "uuid-789",
```

```
 "logs": [
  { "student_id": 123456, "scan_time": "...", "status": "check-in" },
  { "student_id": 654321, "scan_time": "...", "status": "check-in" }
 ]
}
```

6.Composite Design

6.1 Pseudocode Specifications

Pseudo code for verification component

```
function verifyStudent(qrData, invigilatorId, deviceLocation) {
  student = decodeQR(qrData);
  session = getCurrentExamSession(student, deviceLocation);
  if (!session) return {status: "FAIL", reason: "Not registered for this exam"};
  if (alreadyMarked(student, session))
    return {status: "WARNING", reason: "Already marked present"};
  log = createScanLog(student.id, invigilatorId, "verification", session.id);
  markAttendance(student.id, session.id);
  broadcastToDashboard({event: "new_attendance", student, log});
  return {status: "SUCCESS", studentInfo: student};
}
```

Pseudo code to generateExamCard

```
PROCEDURE generateExamCard(studentId: Integer)
  student ← getStudentById(studentId)
  IF student is null THEN RETURN error("Student not found") END IF
  modules ← getRegisteredModules(studentId)
  qrPayload ← {
    studentId: student.id,
    name: student.name,
    modules: modules,
    timestamp: currentTime()
  }
  qrCodeData ← encryptAndEncodeQR(qrPayload)  // Base64 + encryption
  card ← createExamCardRecord(studentId, qrCodeData)
  RETURN {cardId: card.id, qrCode: qrCodeData}
END PROCEDURE
```

Pseudocode for logScriptCollection()

```
PROCEDURE logScriptCollection(studentId: Integer, invigilatorId: Integer, sessionId:
Integer)
```

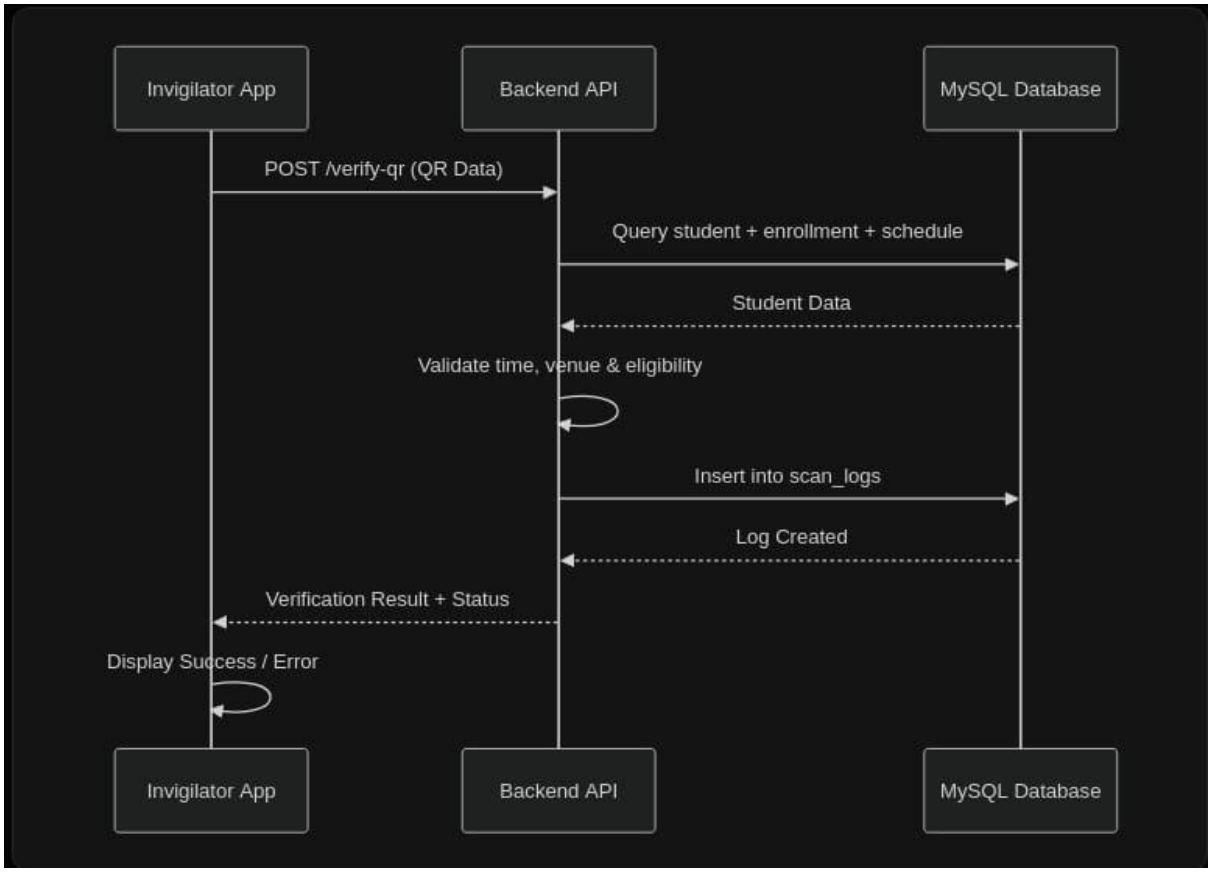
```
  IF NOT isVerifiedToday(studentId, sessionId) THEN
    RETURN error("Student must be verified first")
  END IF
  log ← createScanLog(studentId, invigilatorId, "script_collection", sessionId)
  updateScanLog(log.id, {action: "script_collection", script_status: "collected"})
  broadcastRealTimeUpdate("script_logged", studentId, sessionId)
  RETURN success("Script collection logged")
END PROCEDURE
```

Pseudocode for generateAttendanceReport()

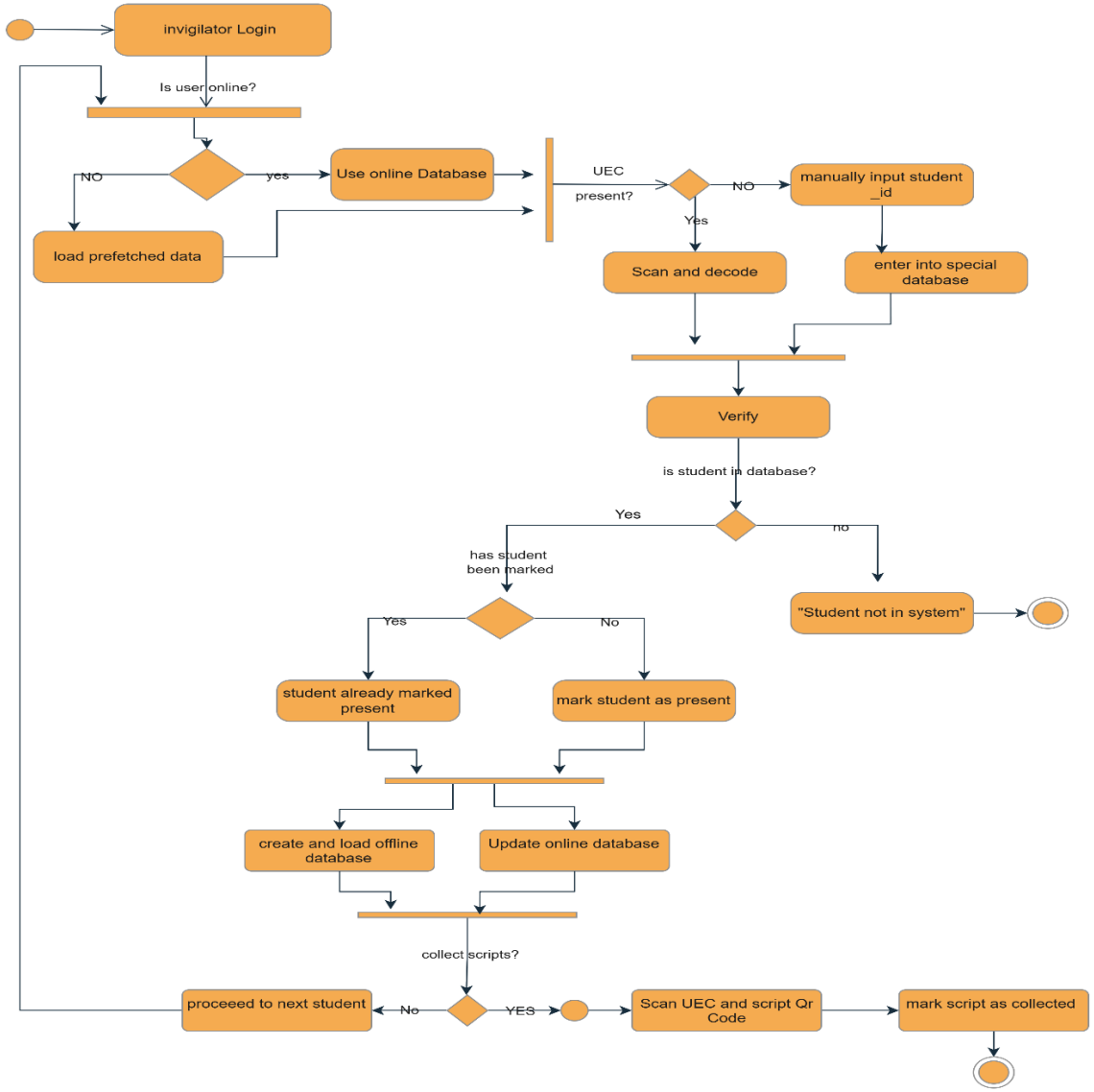
```
PROCEDURE generateAttendanceReport(sessionId: Integer, format: String)
  logs ← queryScanLogsBySession(sessionId)
  summary ← calculateStatistics(logs)  // attendance %, anomalies, etc.
  IF format = "PDF" THEN
    report ← generatePDFReport(summary)
  ELSE IF format = "Excel" THEN
    report ← generateExcelReport(summary)
  END IF
  saveReportToStorage(report)
  RETURN reportLink
END PROCEDURE
```

6.2Behavioural Design

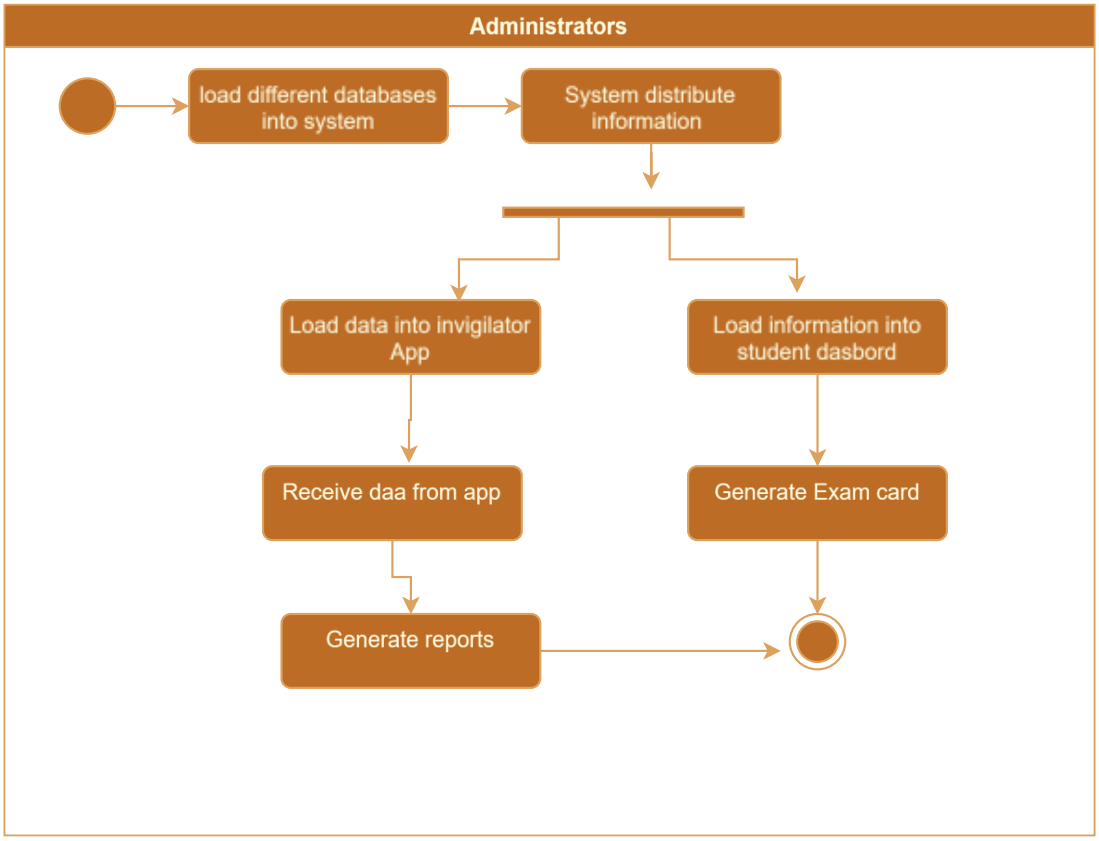
6.2.1Sequence diagram



6.2.2Activity diagram -scan verification and script collection



6.2.3Activity diagram to generate UEC



7.NON-FUNCTIONAL DESIGN CONSIDERATIONS

7.1 Performance and Load Handling

To manage the high concurrency expected during peak examination windows (e.g., thousands of concurrent scans within a 15-minute window), the following strategies are implemented:

- Horizontal Scaling: The Backend API will be containerized using Docker, allowing the system to scale instances dynamically based on CPU/Memory load.
- Database Indexing: Composite indexes will be applied to the STUDENTS(student_id) and EXAM_SCHEDULE(course_code, exam_date) fields to ensure sub-second query response times during verification.
- Connection Pooling: The MySQL database will utilize a connection pool (size: 50–100) to minimize the overhead of frequent handshakes between the API and the database.

7.2 Security and Data Integrity

- Encryption Standards: * Data at Rest: All sensitive data, including student and invigilator passwords, must be hashed using the BCrypt algorithm with a salt factor of 12.
    - Data in Transit: All communication between the Invigilator App and Backend API will be secured via TLS 1.2/1.3 (HTTPS).
- Authentication: API access requires a JSON Web Token (JWT). Tokens are issued upon successful login and must be included in the header of the POST /verify-qr request.

7.3 Offline Mode Design and Synchronization

To ensure the system remains functional in venues with poor connectivity (e.g., deep inside large halls), the following offline architecture is designed:

7.3.1 Local Caching Strategy

The Invigilator App will utilize an SQLite or Room database for local persistence.

- Pre-fetching: 30 minutes before a scheduled exam, the app will automatically download the relevant ENROLLMENT and STUDENT data for that specific VENUE_ID and COURSE_CODE.
- Local Logging: If the Backend API is unreachable, the app will store scan results locally in a PENDING_SYNC table with a timestamp.

7.3.2 Sync Conflict Resolution

When connectivity is restored, the app will initiate a background sync process:

- LWW (Last Write Wins): For attendance status, the record with the most recent timestamp in the SCAN_LOGS will be considered the "truth."
- Duplicate Prevention: The Backend API will enforce a unique constraint on the combination of student_id, course_code, and exam_date to prevent duplicate attendance entries during bulk sync.

7.4 Fault Tolerance and Availability

- Retries: The mobile client will implement an Exponential Backoff retry strategy for failed API calls caused by transient network errors.
- Health Monitoring: A /health endpoint will be implemented on the Backend API to allow load balancers to redirect traffic away from unhealthy instances.

8. HUMAN INTERFACE DESIGN

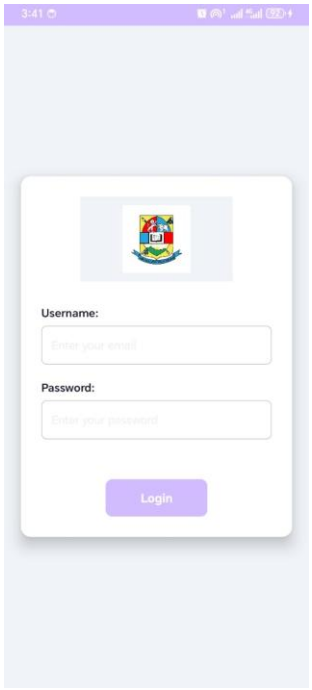
8.1 Overview of User Interface

Android App (Invigilator): Simple, scanner-centric interface with large buttons and immediate visual feedback (green/red).

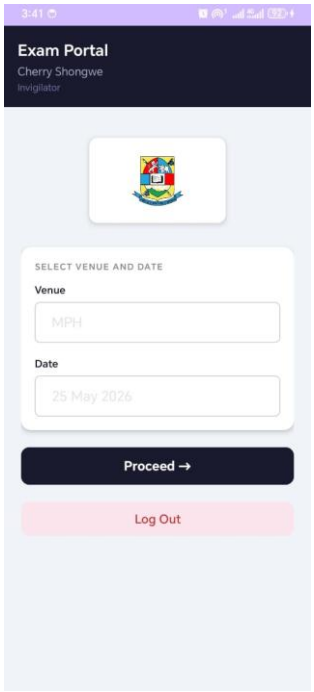
Web Dashboard: Real-time overview with charts showing attendance per venue/course, anomaly alerts, and report generation.

Users can complete all features: scan students, log scripts, view live statistics, and export reports.

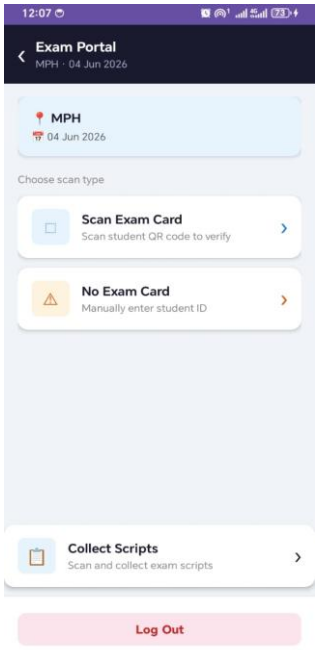
8.2 Screen Images



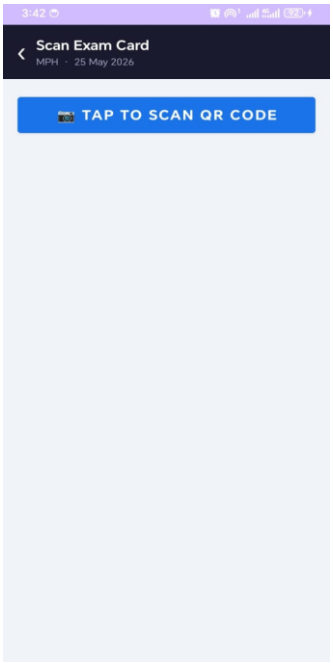
This is the login interface for the invigilator app



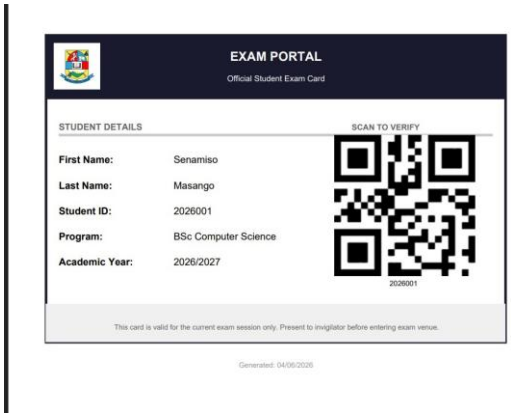
Once logged in the invigilator will then input the date and venue they are in



This interface allows the invigilator to pick whether to mark attendance or collect scripts

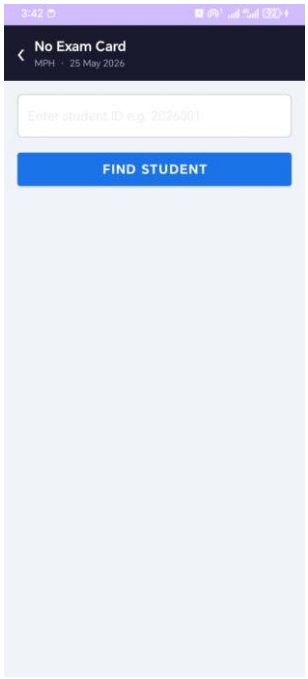


If the invigilator chooses to scan students this will appear

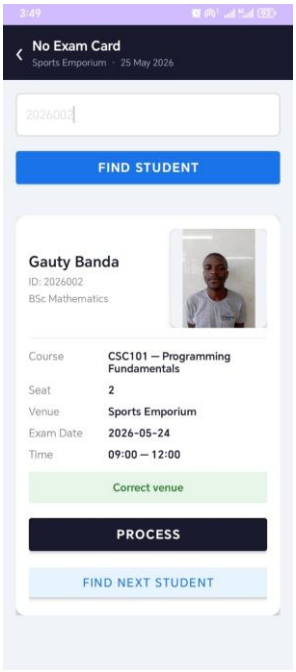


This is the UEC that a student a student will present to invigilator

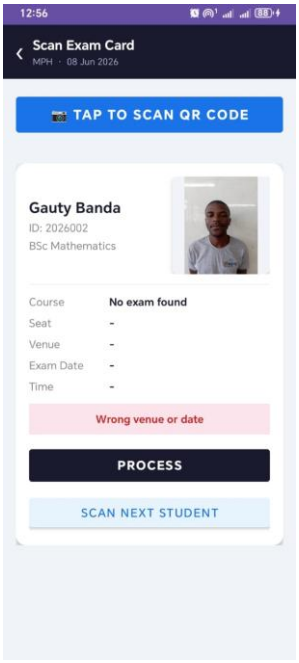
Upon scanning the UEC, the above will appear that the invigilator will use to verify if it truly is the student and mark them



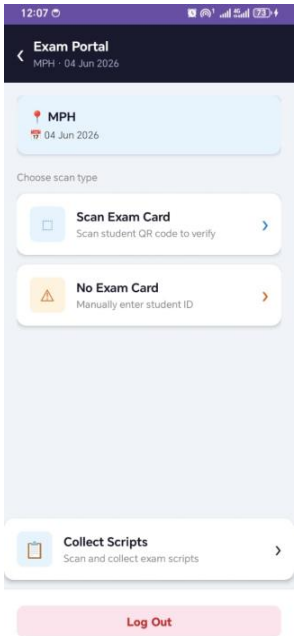
If the student does not have their UEC with them, the invigilator can manually verify them using the no exam card option



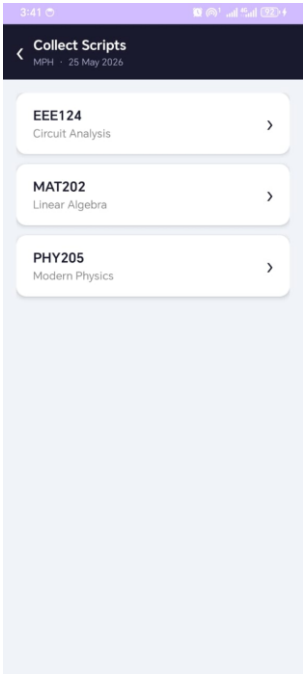
Using the student ID and the retrieved information, the invigilator will verify the student



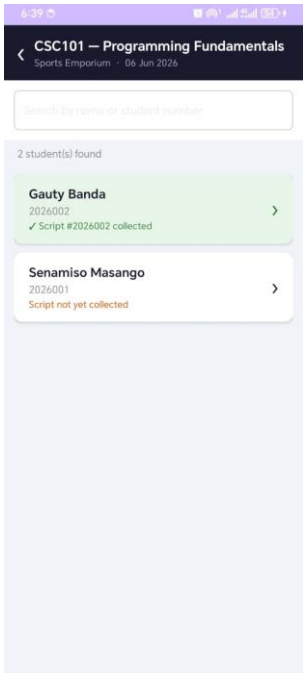
If the wrong student is at an examination the app will notify the invigilator



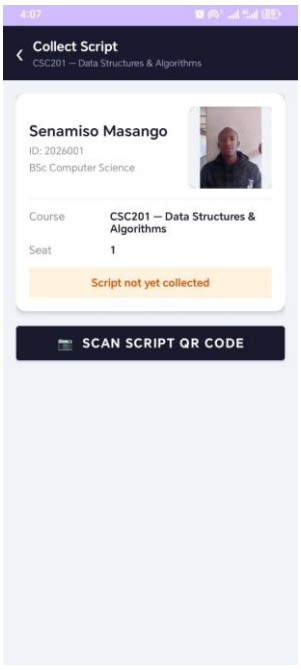
Upon finishing processing a student, the app will take them back and give them the options above again



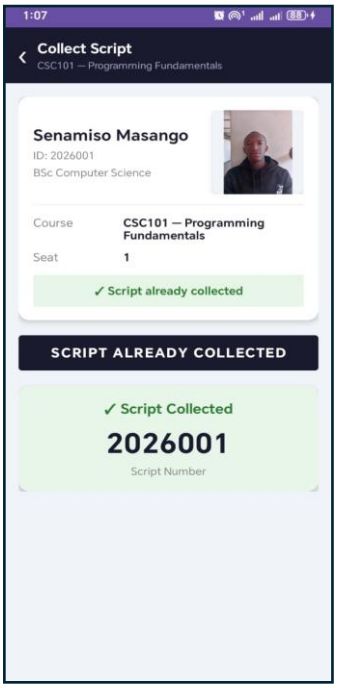
When the invigilator chooses the collect script option, they can then choose the course whose scripts they will collect



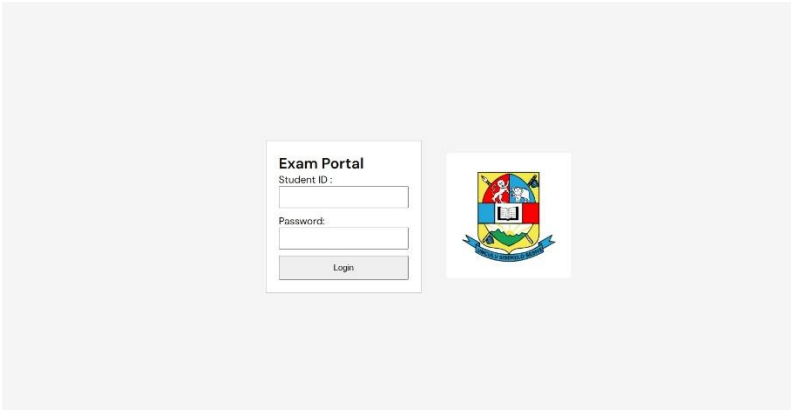
Invigilator can keep track of collected scripts



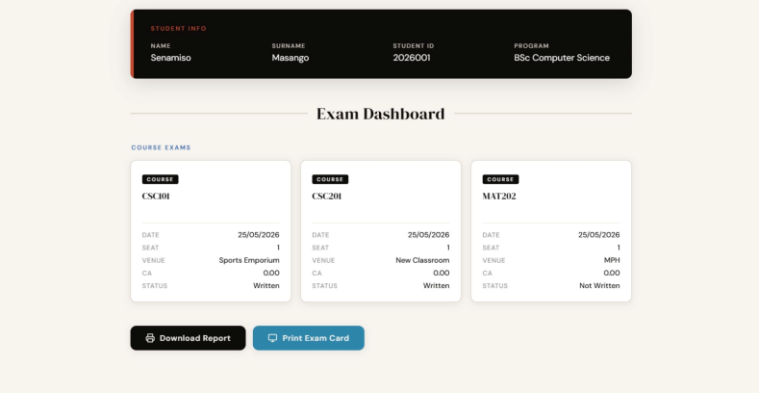
The invigilator will scan the UEC of the students whose script is to be collected



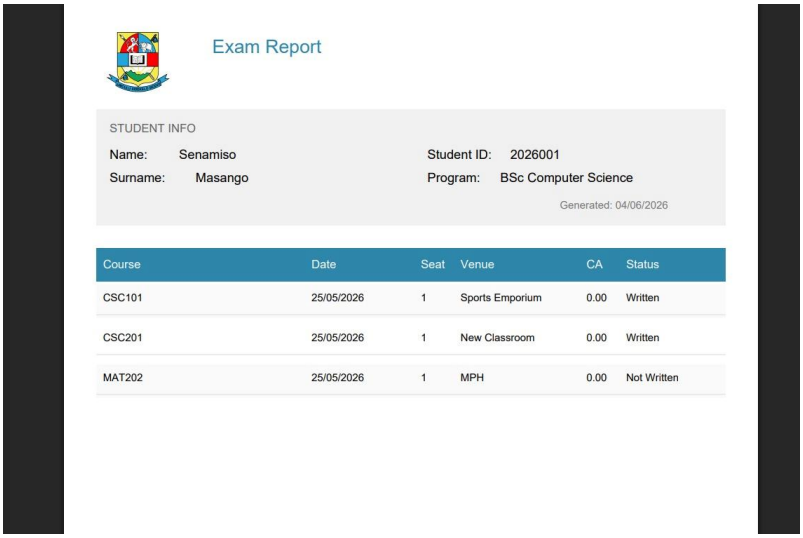
Once the script is collected it will reflect on the app and the invigilator will proceed to the next student



This is the login interface to the dashboard for the students



Students can view status of their collected paper, papers they will write and generate an emergency exam card



This is an example of the student exam report that can be downloaded and act as proof that student scripts were collected and they wrote the exam

8.3 Screen Objects and Actions

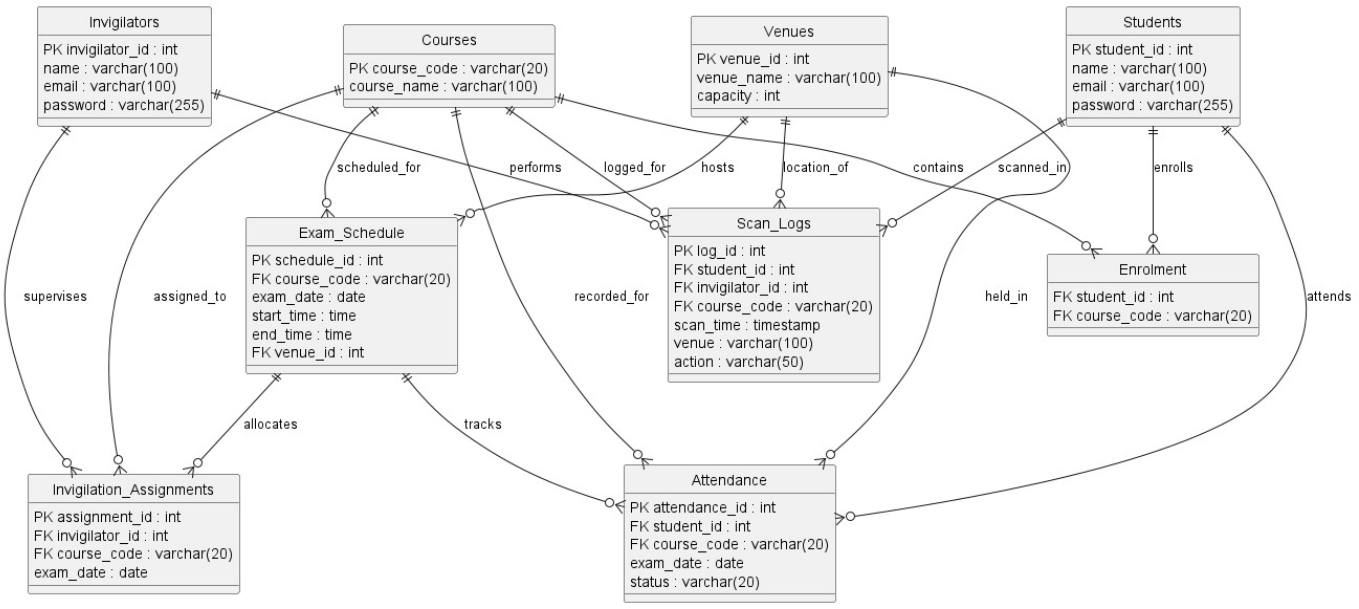
- **Scan Button**: Activates camera and triggers verification flow.
- **Result Banner**: Displays success/error with student details.
- **Dashboard Cards**: Clickable widgets for detailed views.
- **Export Button**: Generates downloadable reports.

9. REQUIREMENTS MATRIX

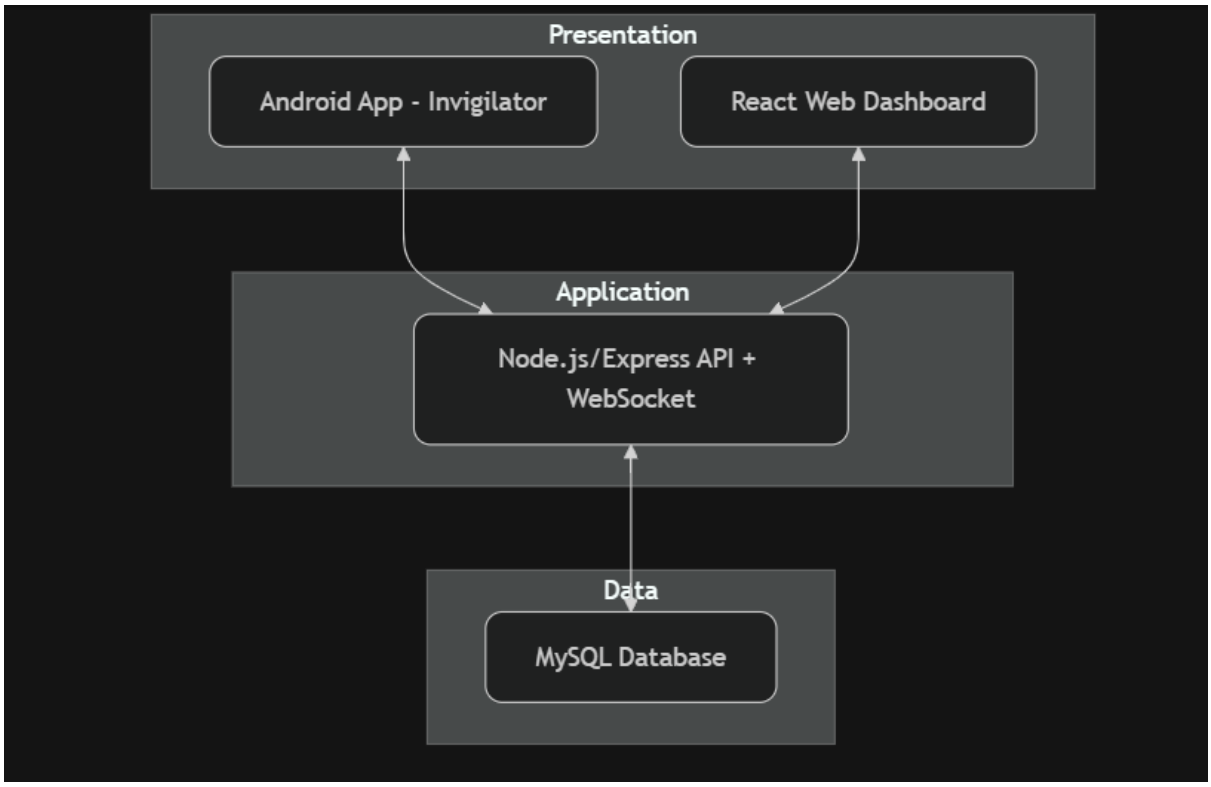
SRS ID	Requirement Description	SDD Components / Modules
FR-01	Universal Exam Card with QR code	Exam Card Management + QR Service
FR-02	User Authentication (JWT)	Authentication Subsystem
FR-03	Identity Verification via Android	Verification & Logging Subsystem
FR-04	Automatic Attendance Marking	Scan Log + Attendance Module
FR-05	Script Collection Logging	Scan Log (script_collection action)
FR-06	Real-time Dashboard	Monitoring Subsystem + WebSocket
FR-07	Reporting	Reporting Service

10. APPENDICES

Appendix A: Detailed ER Diagram



Appendix B: Architecture Diagram



Appendix C: Sequence Diagrams